

FEDERATED LEARNING PRIVACY AND SECURITY ENHANCEMENT

Oscar Chen

Department of Computer Science
Golisano College of Computing and Information Sciences
Rochester Institute of Technology
Rochester, NY 14586
oyc2709@rit.edu

Abstract—Federated Learning (FL) has emerged as a promising alternative to centralized machine learning by enabling collaborative model training without direct data sharing. With recent advances in FL on feature analysis, aggregation algorithms, privacy, and security, FL improves data locality and lowers raw data exposure compared to centralized learning. However, it introduces new methods of attacks such as model poisoning, inference/membership attacks and important tradeoffs among communication cost, robustness, and privacy guarantees. We compare classical techniques such as federated averaging with robust aggregation strategies including Trimmed Mean, Krum, Bulyan, and Robust Federated Aggregation, highlighting trade-offs in computational efficiency, scalability, and resilience against compromised clients. Our analysis shows how FL improves data privacy through secure aggregation and differential privacy, and how it deals with threats such as poisoning and inference attacks. A survey of recent patents shows developing interest in communication efficiency, adaptive aggregation and privacy solutions. Overall, FL demonstrates strong potential for privacy-first machine learning, but faces challenges in robustness, communication overhead, and data heterogeneity that define areas of interest for future research

I. EXPERIMENTAL

A. Objectives

- Implement an aggregation algorithm using Proportional-Integral-Derivative (PID) based client filtering with the Flower Framework
- Detect and automatically exclude poisoned clients based on deviations in model parameters.
- Evaluate the effect of the PID defense on model loss and accuracy.
- Develop visualization tools for analysis

B. Defense Algorithm

The PID (Proportional-Integral-Derivative) mechanism was integrated into the custom aggregation strategy based on FedAvg. For each round r , the server computes a centroid model from the participating clients and measures each clients normalized L2 distance, d , to this centroid.

Round 1: $PID = K_p * d$

Round $r \geq 2$: $PID = K_p * d + K_i * \text{SUM}(d + K_d * (d - d_{\text{prev}}))$

Clients with PID greater than the set threshold are flagged as anomalous and permanently excluded from future rounds. For this experiment, parameters were initialized as $K_p = 5$, $K_i = 0.5$, $K_d = 1$, and the threshold = 2.5. These values were obtained through trial and error after starting with $K_p = 1$, K_i & $K_d = 0$, and the threshold at 1.0.

C. Setup

Dataset: CIFAR-10 (10 classes, 50,000 images). The dataset was partitioned into 20 clients (around 2500 samples each), split into 80% train and 20% test.

Two clients (3 and 17) were selected to be poisoned through label flipping from Part 2 to simulate poisoned clients.

Training parameters:

Global Rounds: 5

Training Epochs: 10

Batch Size: 32

Learning Rate: 0.5

Optimizer: Stochastic Gradient Descent

II. RESULTS

A. History of loss per client over rounds

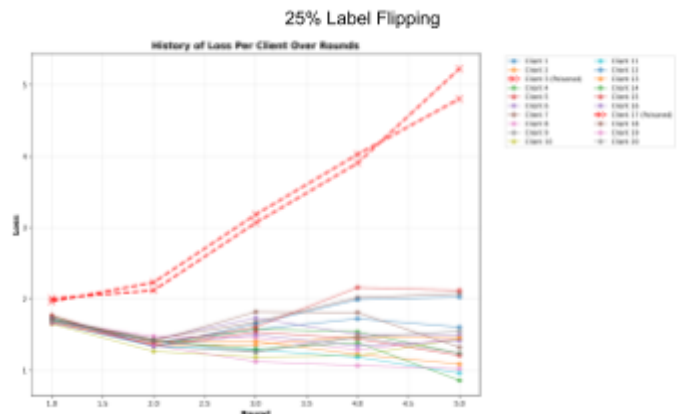


Fig. 1. Individual loss per client for 25% label flipping (red dotted lines are poisoned clients)

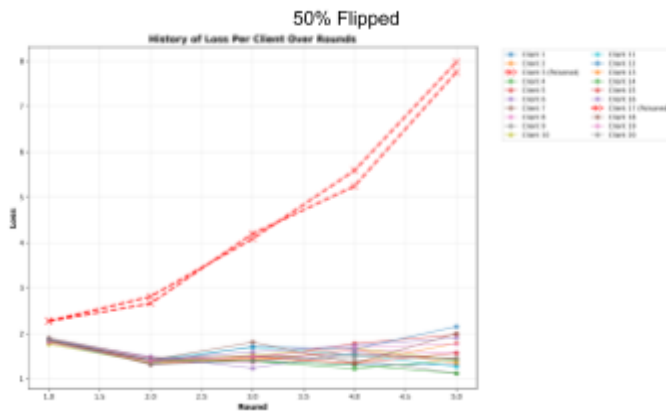


Fig. 2. Individual loss per client for 50% label flipping (red dotted lines are poisoned clients)

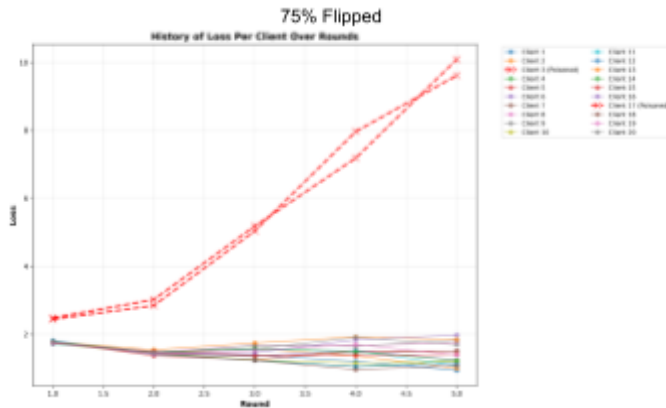


Fig. 3. Individual loss per client for 75% label flipping (red dotted lines are poisoned clients)

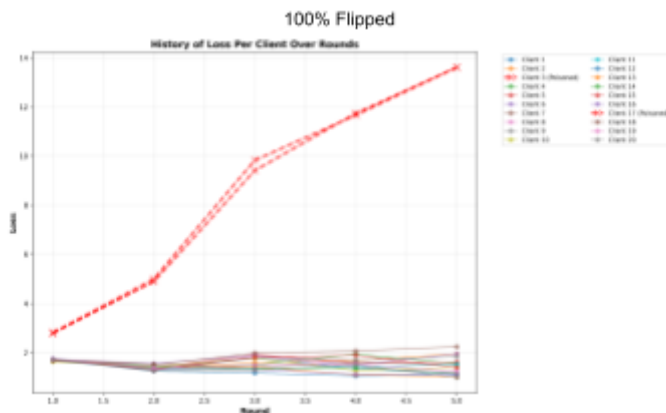


Fig. 4. Individual loss per client for 100% label flipping (red dotted lines are poisoned clients)

According to the figures above (Fig. 1-4) Loss is clearly distinguishable and rises quicker as the percentage of

poisoning increases. While regular clients tend to stay around a similar loss range.

B. History of average loss among clients over rounds

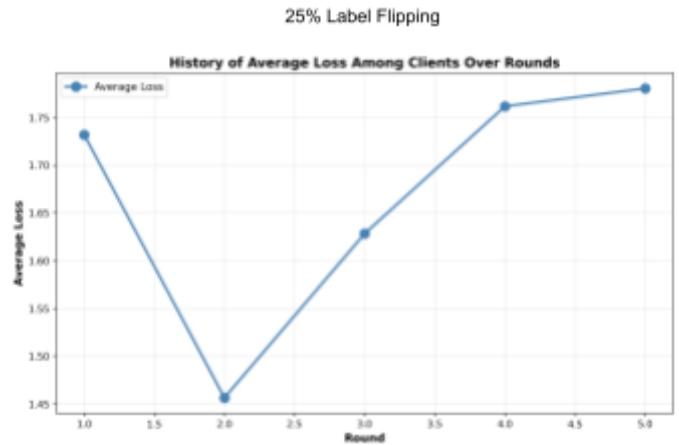


Fig. 5. Average loss per round for 25% label flipping.

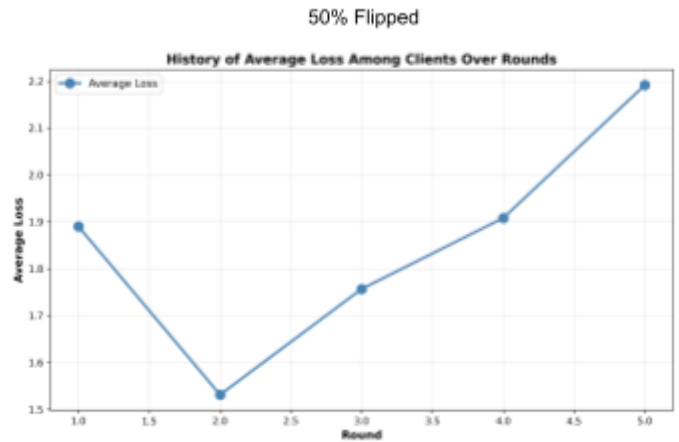


Fig. 6. Average loss per round for 50% label flipping.

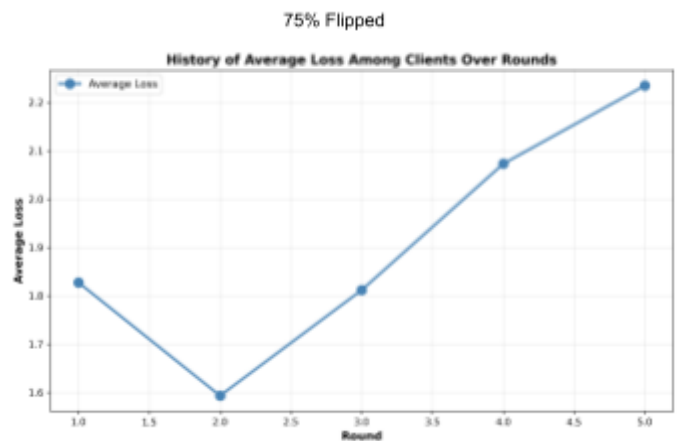


Fig. 7. Average loss per round for 75% label flipping.

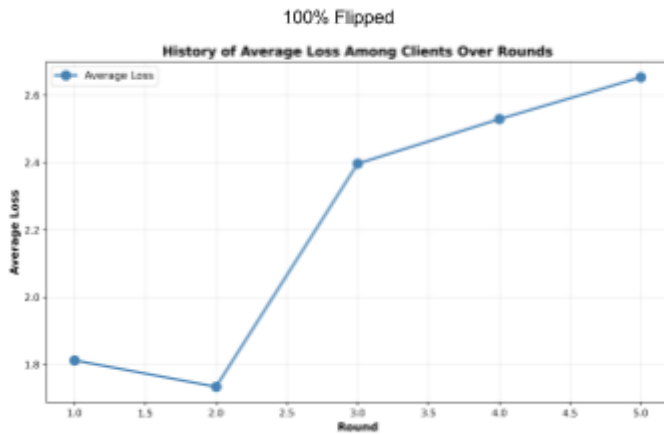


Fig. 8. Average loss per round for 100% label flipping.

The average loss among clients over each round. As shown in the figures above (Fig. 5-8) by round 3 the poisoned clients are almost guaranteed to have been included in a train cycle which is where the loss spikes up the most and why loss is sometimes lower in the first two rounds because the poisoned clients haven't been trained on.

C. History of evaluation accuracy per client over rounds

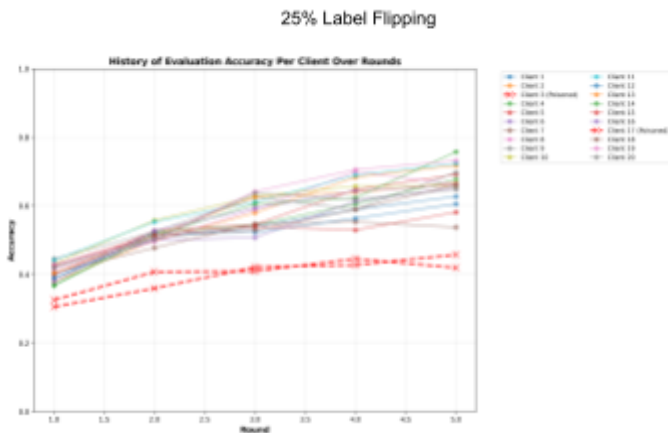


Fig. 9. Accuracy per client over each round for 25% label flipping (red dotted lines are the poisoned clients).

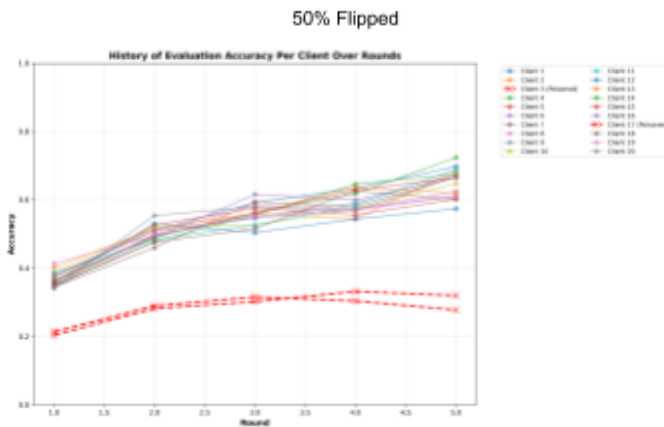


Fig. 10. Accuracy per client over each round for 50% label flipping (red dotted lines are the poisoned clients).

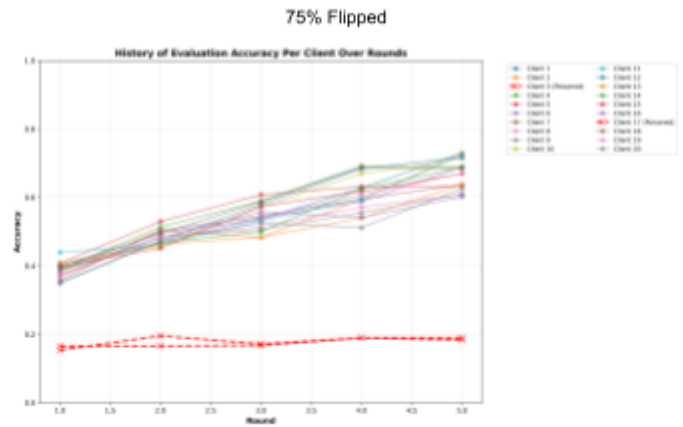


Fig. 11. Accuracy per client over each round for 75% label flipping (red dotted lines are the poisoned clients).

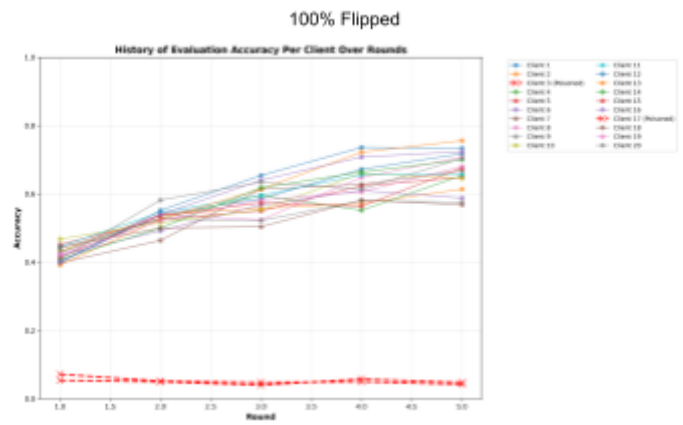


Fig. 12. Accuracy per client over each round for 100% label flipping (red dotted lines are the poisoned clients).

As shown in the figures above (Fig. 9-12) the accuracy per client is noticeably lower for poisoned clients (3 and 17) and the accuracies are much lower as the percentage of poisoning increases.

D. History of average evaluation accuracy among clients over rounds

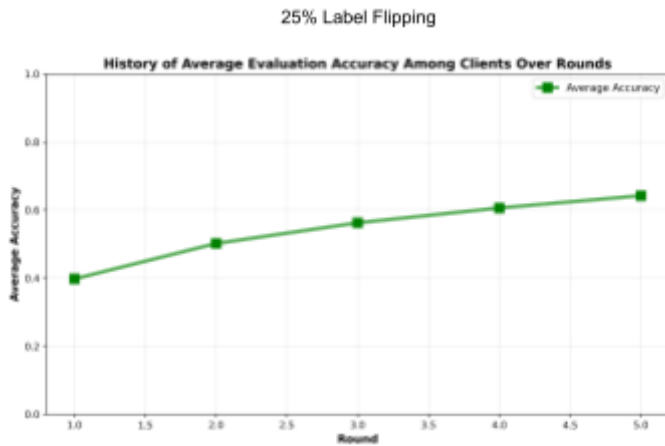


Fig. 13. Average accuracy per round for 25% label flipping

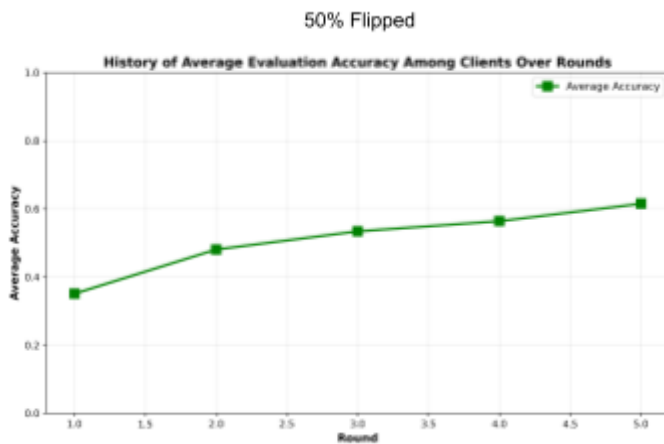


Fig. 14. Average accuracy per round for 50% label flipping

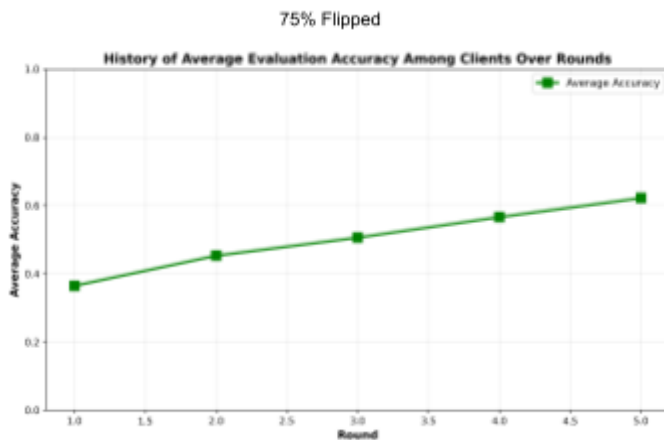


Fig. 15. Average accuracy per round for 75% label flipping

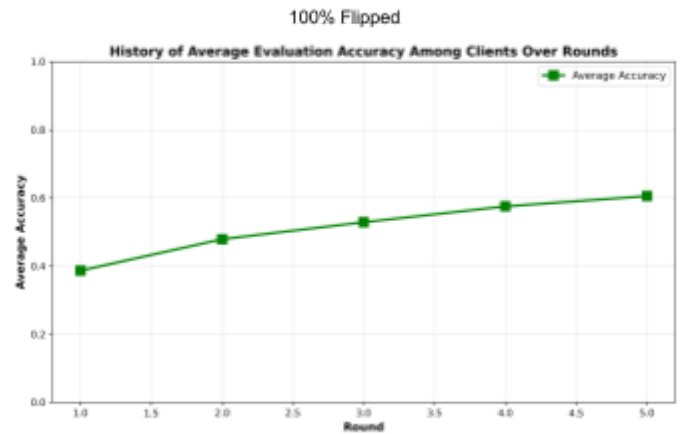


Fig. 16. Average accuracy per round for 100% label flipping

As shown in the figures above (Fig. 13-16) the average accuracy per round increases at a stable rate and compared to the graphs in part 2, is more stable and experiences smaller drops even with high percentages of poisoning.

E. Loss and accuracy dependence on the attack severity

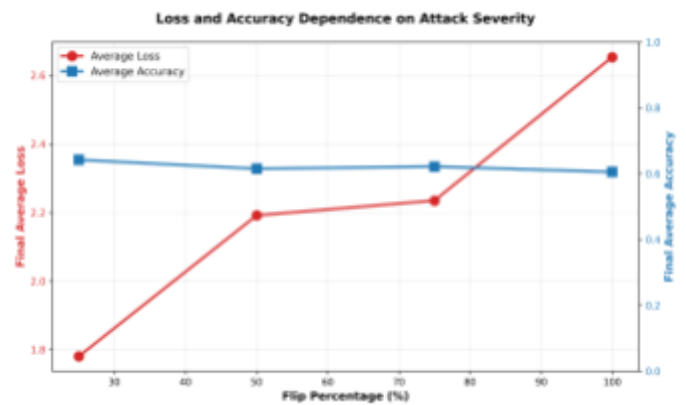


Fig. 17. The loss and accuracy depending on the percentage of poisoning (label flipping attack)

As shown in the figure above (Fig. 17) the average loss increases with the percentage of poisoning and the average accuracy decreases with the percentage of poisoning, but this is most likely due to the effect of higher percentages of poisoning before they were excluded, however, with more rounds of training it is likely that the accuracy would increase.

F. Client Removal Dynamics

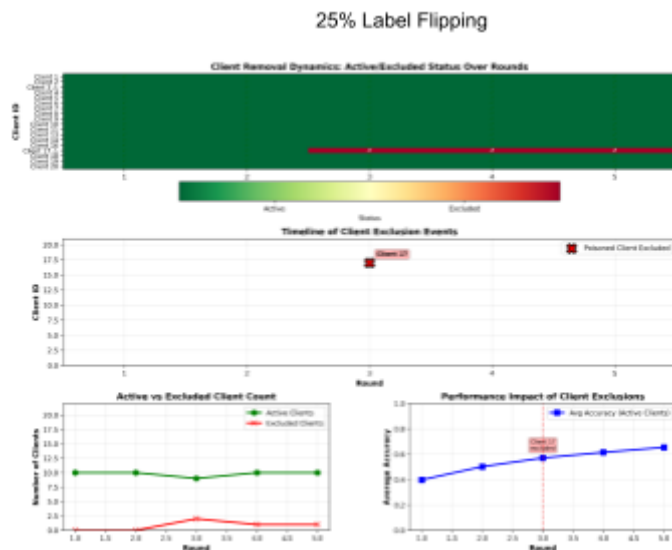


Fig. 18. Client removal dynamics for 25% label flipping

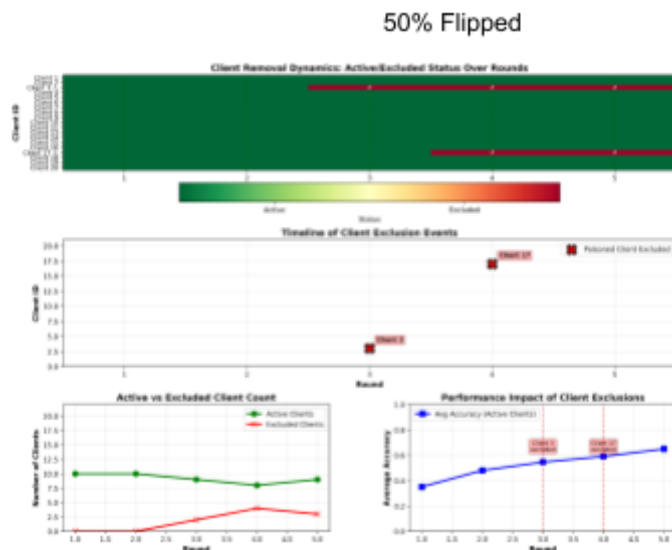


Fig. 19. Client removal dynamics for 50% label flipping

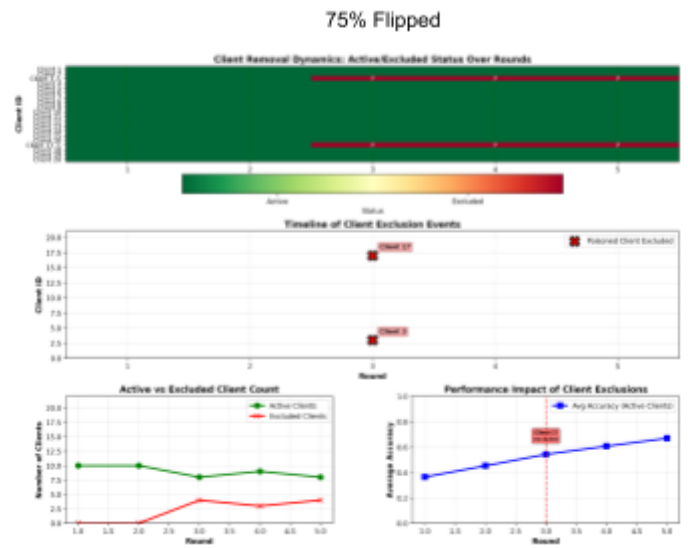


Fig. 20. Client removal dynamics for 75% label flipping

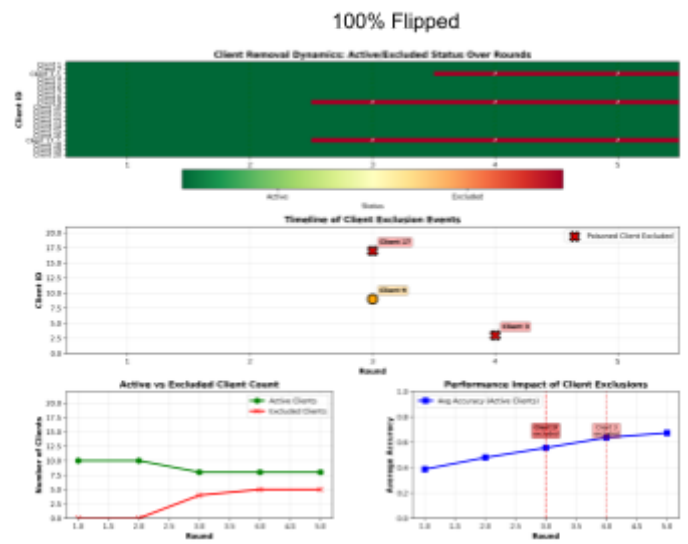


Fig. 21. Client removal dynamics for 100% label flipping

The figures above (Fig. 18-21) show 4 graphs an Active/Excluded Status Heatmap (Top Panel) which displays the participation status of all 20 clients across training rounds, the Timeline of Exclusion Events (Middle-Left Panel) the scatter plot marks the exact rounds when clients were excluded, the Active Client Count (Bottom-Left Panel) The line graph tracks the number of active vs excluded clients over rounds, due to how the data is stored duplicate values may have been included, finally, the Performance Impact (Bottom-Right Panel) The plot shows the average accuracy of active clients over rounds, with vertical red lines marking exclusion events which show that after exclusion the accuracy do indeed increase normally which shows that the PID client filtering is indeed effective. However, in the 100% label flipped simulation there is a false positive (excluded non-poisoned client). This means that if there are too many poisoned clients or clients with too much poisoning, non-malicious clients may fall victim to the PID filtering.

Additionally, if there are poisoned clients with weaker attacks, the PID filter may not notice the weaker attacks, so in order to address these issues we would need to dynamically update the PID values to better detect poisoned clients.

III. WORK DISTRIBUTION

I worked alone on this project although I did use AI to help with graph generation, metric collection, and optimizations with models and data parsing. Running the simulations was definitely the most time consuming, taking over a week as I did have to increase the number of clients run per train cycle from 4 to 10 in order to produce more differences in the PID values. The second longest part was when I tried my best to fine tune the K values which took roughly 2 days as I did have to wait for the simulation to finish before I could analyze and update the values. Unfortunately, I was not able to increase the average accuracy above ~63%